



UNIVERSITATEA TEHNICĂ „GHEORGHE ASACHI” DIN IAȘI
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
Domeniul: Calculatoare și Tehnologia Informației
Programul de studii: Tehnologia Informației



PROIECT DE DIPLOMĂ

Coordonator științific:
Conf. dr. ing. Paul-Corneliu
HERGHELEGIU

Absolvent:
Raimond-Eduard BUTNARU

Iași, 2025



UNIVERSITATEA TEHNICĂ „GHEORGHE ASACHI” DIN IAȘI
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
Domeniul: Calculatoare și Tehnologia Informației
Programul de studii: Tehnologia Informației



Metode de analiză a infrastructurii rutiere utilizând computer vision

PROIECT DE DIPLOMĂ

Coordonator științific:
Conf. dr. ing. Paul-Corneliu
HERGHELEGIU

Absolvent:
Raimond-Eduard BUTNARU

Iași, 2025

Cuprins

1	Introducere	1
1.1	Obiectivele lucrării	1
1.2	Metodologia de cercetare utilizată	2
1.3	Structura generală a lucrării	2
2	Fundamentarea teoretică și documentarea bibliografică	3
2.1	Documentarea algoritmilor utilizați	3
2.2	Algoritmi menționați	7
3	Soluția propusă	9
3.1	Problema prezentată	9
3.2	Tehnologii utilizate	9
3.3	Operații de pregătire a datelor	9
3.4	Implementarea operațiilor preliminare	11
3.5	Implementarea logicii de detecție, urmărire și numărare a vehiculelor	15
3.6	Compararea cu un model YOLOv8	17
4	Testarea aplicației și rezultate experimentale	19
4.1	Rezultatele antrenării modelului	19
4.2	Testarea celor 2 metode	19
4.3	Rezultate	20
5	Concluzii	25
5.1	Gradul de realizare al temei propuse	25
5.2	Dificultăți întâmpinate	25
5.3	Comparația cu alte proiecte similare	25
5.4	Posibile direcții de dezvoltare	26
	Bibliografie	27
	Anexe	29
1	Codul funcției <i>remove_cars</i>	29
2	Codul funcției <i>warmup_mog</i>	29
3	Codul constantelor pentru zona de interes	30
4	Codul constantelor utilizate pentru definirea intrărilor și ieșirilor din intersecție	30
5	Codul utilizat pentru crearea măștii zonei de interes	31
6	Codul de preprocesare a cadrelor	31
7	Codul clasei <i>EnhancedCentroidTracker</i>	32
8	Secțiunea de cod pentru detecție	35
9	Codul utilizat pentru urmărirea fiecărui obiect identificat	35
10	Bucula principală de detecție utilizând machine learning	35

11	Rezultatele obținute în urma antrenării	37
----	---	----

Metode de analiză a infrastructurii rutiere utilizând computer vision

Raimond-Eduard BUTNARU

Rezumat

În municipiul Iași, sunt aproximativ 100 de intersecții, dintre care mai bine de jumătate sunt aglomerate aproape zilnic; această aglomerație este cauzată atât de șoferii nerăbdători, cât și de proiectarea improprie a intersecțiilor.

Motivația acestei lucrări a apărut din dorința de a crea un mediu sigur și ușor de utilizat pentru a testa proiectarea intersecțiilor înainte de construcția acestora, pentru a reduce numărul de accidente cauzate de infrastructură.

Pentru dezvoltarea acestei lucrări, am ales ca punct de pornire intersecția din Podu Roș, aceasta este o intersecție complicată până și pentru localnici. Forma ei de rond induce în eroare pe toată lumea cum că ar putea fi un sens giratoriu, chiar și în sistemele de navigare, această intersecție este considerată ca o intersecție cu sens giratoriu. În realitate, este o intersecție cu multe reguli ciudate, care doar întâmplător are formă de rond.

Obiectivul principal al acestei lucrări este analizarea unor date statistice ale acestei intersecții utilizând metode de *computer vision* pentru a pune în evidență detaliile ce nu doar că îngreunează traficul în municipiul Iași cât și în România. Aceste detalii pot fi o densitate a traficului foarte mare, manevre periculoase executate de conducătorii auto sau chiar și proiectarea improprie a întregii intersecții.

Pentru a îndeplini acest obiectiv, vor fi prezentate în această lucrare 2 metode comparate între ele. Prima, soluția propusă în această lucrare, iar a doua, o metodă simplă ce folosește un model YOLOv8 de învățare automată.

Soluția propusă în această lucrare constă într-un set de pregătire a înregistrării, metode de preprocesare, operații morfologice ce evidențiază elemente cheie și șterg elemente inutile și o metodă de urmărire a vehiculelor.

Capitolul 1. Introducere

În 2023 Ministerul Afacerilor Interne ¹a înregistrat un număr de 4526 de accidente rutiere, din care 3536 au fost soldate cu victime grav rănite, iar 1545 din aceste accidente au rezultat cu decesul persoanelor implicate. Conform aceleași surse, principalele cauze ale accidentelor sunt:

1. Viteza neadaptată la condițiile de drum. Principalul motiv pentru care limitele de viteză nu sunt respectate este comparația cu infrastructura din Europa centrala și de vest. Spre exemplu: parcurgem un număr de kilometri similar cu germanii doar că în timp aproape dublu în condiții de trafic cu lipsa aglomerației. Pentru referință, în Germania, drumul dintre Stuttgart și München, care are o distanță de 232 de kilometri, are un timp de parcurgere mediu de 2 ore și 30 de minute. La noi în România, un drum de 128 de kilometri între Bacău și Iași durează, într-o zi fără trafic intens, aproximativ 2 ore și 15 minute, timp foarte mult pentru distanță mult mai mică față de alte zone.
2. Traversarea neregulamentară a pietonilor indiferent dacă este realizată pe treceri special amenajate sau nu.
3. Neacordarea de prioritate a pietonilor față de vehiculele participante la trafic.
4. Neacordarea de prioritate a bicicliștilor față de ceilalți participanți la trafic sau bicicliști aflați în stare de ebrietate.
5. Neacordarea de prioritate către alte vehicule. Avem prea multe intersecții unde drumul fără prioritate are o vizibilitate extrem de redusă față de drumul cu prioritate, motiv pentru care șoferii care doresc să vadă pentru a se asigura, ies din nefericire și taie calea altor persoane fără a avea o intenție rea în executarea manevrei.

În județul Iași, bilanțul local contribuie în mod negativ la bilanțul național, conform datelor publicate de Inspectoratul de Poliție Județean Iași. În 2024, au fost înregistrate 1.389 de accidente rutiere, în urma cărora au rezultat 1.575 persoane ușor rănite, 107 rănite grav și 56 de decese. Din aceste date, se menționează că 7 persoane ar fi murit în municipiul Iași în urma accidentelor rutiere, iar 36 au fost rănite grav.

1.1. Obiectivele lucrării

1.1.1. Obiectivele generale ale lucrării

Într-un context general, această lucrare are ca scop analiza intersecțiilor și a traficului din acestea pentru a facilita o proiectare cât mai eficientă a acestora. Astfel, se aduce la cunoștință și un grad de incompetență a personalului responsabil de drumurile din România. Totodată, în contextul intersecției Podu Roș din Iași, un alt obiectiv al lucrării este identificarea intrărilor, ieșirilor și a porțiunilor de drum din cadrul intersecției ce împiedică un curs normal al traficului. În această categorie intră și drumuri de intrare sau de ieșire supraaglomerate, puncte ale intersecției unde șoferii se încadrează necorespunzător, dar și elemente ce încurcă traficul mai mult decât îl ajută.

¹Statistici 2023: <https://politiaromana.ro/ro/structura-politiei-romane/unitati-centrale/directia-rutiera/statistici>

1.1.2. Obiectivele specifice ale lucrării

Obiectivele lucrării în contextul de *computer vision* tind spre implementarea unui algoritm ce detectează, etichetează și urmărește vehiculele prezente în intersecție. Scopul final al acestui algoritm este extragerea unor metrici ce vor pune în lumină atât eficiența intersecției, cât și eficiența algoritmului implementat.

1.2. Metodologia de cercetare utilizată

Metodologia de cercetare principală, utilizată în această lucrare, se bazează pe o abordare aplicată și cantitativă, folosind algoritmi de analiză video pentru a extrage automat valori din traficul înregistrat cu drona.

Datele video sunt pregătite, preprocesate și prelucrate prin tehnici de segmentare, detectare și urmărire, rezultând un set de indicatori, cum ar fi numărul de vehicule care călătoresc într-o perioadă de timp, direcțiile de deplasare a vehiculelor, densitatea traficului și posibilitatea de a calcula punctele de congestie ale acestei intersecții.

Simultă, acest proiect folosește și o abordare bazată pe metode computaționale, cu un aspect practic pentru algoritmi eficienți

1.3. Structura generală a lucrării

- Rezumatul lucrării
- 1. Introducere
- 2. Fundamentarea teoretică și documentarea bibliografică
- 3. Soluția propusă
- 4. Testarea aplicației și rezultate experimentale
- 5. Concluzii
- Bibliografie
- Anexe

Capitolul 2. Fundamentarea teoretică și documentarea bibliografică

În figura 2.1 se poate observa un cadru din filmarea ce îmi va folosi ca set de date. Înregistrarea aparține echipei *Aerial Strike*², o echipă ce se specializează în filmări utilizând drone. Următoarele fundamente teoretice stau la baza soluției propuse pentru atingerea obiectivului de detecție a vehiculelor ce parcurg această intersecție.



Figura 2.1. Cadru din filmarea originală

2.1. Documentarea algoritmilor utilizați

2.1.1. Filtrul median

Conform [1], filtrul median a fost limitat de tehnologia pe care a fost dezvoltat în mod inițial, însă odată cu înaintarea în tehnologie și creșterea puterii de procesare, utilizările acestui filtru au fost extinse spre noi dimensiuni. Biradar vorbește despre începuturile filtrului median ca un filtru ce diminuează zgomotul, nicidecum nu s-ar fi gândit cineva că ar putea ajunge să șteargă obiecte cu totul. Modelul propus de Biradar este al unui filtru median recursiv utilizat în conjunctură cu piramida Gaussiană. Practic, acest model ar difuza valoarea mediană a pixelilor regiunii țintă dinspre exteriorul acesteia înspre regiunea țintă. În aceeași lucrare, se discută și despre variantele anterioare ale filtrului median. În principiu, când se dorește eliminarea unui obiect dintr-un cadru sau dintr-un set de cadre, se presupune că fundalul este omogen și nu există un alt obiect în spatele obiectului țintă care să necesite reconstrucție. Acest algoritm este testat pe o varietate de cadre pentru a investiga performanța lui în comparație cu algoritmul propus de Bertalmio [2], cu un algoritm bazat pe FoE [3] și cu un algoritm bazat pe piramida Gaussiană [4]. Nu există o metodă standard de evaluare a metricilor unui algoritm de ștergere a obiectelor, așadar evaluarea a fost realizată pe baza *peak signal to noise ratio* sau **PSNR** și este definit după cum se poate observa în (2.1), unde RMSE este calculat în (2.2), $\mathbf{u}$ este imaginea originală, $\mathbf{v}$ este imaginea de ieșire, iar $\mathbf{M} \times \mathbf{N}$ reprezintă dimensiunile pentru $\mathbf{u}$ respectiv $\mathbf{v}$.

$$PSNR = 20 \log_{10} \left(\frac{255}{RMSE(u, v)} \right) \quad (2.1)$$

$$RMSE = \sqrt{\frac{\sum_{i,j} (u_{i,j} - v_{i,j})^2}{M * N}} \quad (2.2)$$

²Aerial Strike webpage: <https://aerialstrike.weebly.com/>

Pentru a calcula PSNR, în mod artificial se adaugă un obiect sau *subimage* peste imaginea originală pentru a crea o nouă imagine cu obiecte străine. Astfel, în urma aplicării algoritmilor, obiectul adăugat este șters și se creează un fundal omogen. Rezultatele se regăsesc în tabelul 2.1 în urma rezultatelor obținute după 2.500 de iterații (acolo unde este cazul).

Tabelul 2.1. Rezultatele testelor

<i>Nr. Crt.</i>	<i>Algortim</i>	<i>PSNR (dB)</i>
1	Bertalmio	31.0438
2	FoE	37.6973
3	Piramida Gaussiană	30.3202
4	Biradar	58.8285

Date fiind aceste metrice, se poate observa cum algoritmul propus de Biradar este mult mai avansat decât iterațiile anterioare.

Această variantă de filtru median a fost utilizată în partea de pregătire a setului de date pentru a obține o înregistrare în care să rămână doar fundalul.

2.1.2. Diferențe absolute

În lucrarea [5] se discută despre o nouă metodă de detecție a mișcării utilizând diferența diferențelor absolute. Principala dezbatere în acest articol pune în opoziție algoritmul de diferențe a diferențelor absolute (DAD ³) față de suma diferențelor absolute (SAD ⁴). Metoda SAD este utilizată în principal pentru detecția mișcării și pentru comprimarea videoclipurilor. Această metodă practic face diferența absolută între pixelii a două cadre consecutive, cu această procedură fiind repetată pe un set mai mare de cadre consecutive, rezultând apoi, prin însumarea acestor diferențe, o matrice SAD 2D finală. Algoritmul urmează ca principiu formula (2.3).

$$SAD = \sum_{i=1}^N |I_1(i) - I_2(i)| \quad (2.3)$$

- I_1, I_2 sunt blocurile comparate
- i este indexul pixelului
- N este numărul total de pixeli în bloc

Pe de altă parte, algoritmul DAD acționează ca diferențele absolute între 2 matrici SAD în conformitate cu formula (2.4).

$$DAD(x, y) = |SAD(x) - SAD(y)| \quad (2.4)$$

- x și y sunt poziții adiacente.

Diferențele absolute au apărut ca și o completare la extragerea prim-planului, în contextul lucrării prezentate, utilizând MOG2, deoarece vehiculele staționează un timp suficient de mult ca acestea să fie considerate fundal. Așadar, utilizând diferențele absolute, putem menține detecția și în momentele de staționare a vehiculelor.

³DAD - Difference of Absolute Differences

⁴SAD - Sum of Absolute Differences

2.1.3. Background subtractor

În lucrarea [6], Marcomini pune în lumină diferențele dintre algoritmi de extragere a fundalului a algoritmului MOG (Mixture of Gaussian) și MOG2. MOG preia un amestec de k modele de distribuții Gaussiene pentru fiecare pixel fundal cu valori pentru k de la 3 la 5. Această metodă a fost propusă inițial în [7], bazat pe [8]. Autorii presupun că fiecare distribuție reprezintă fiecare culoare de fundal și de prim plan. Așadar, ponderea fiecărei distribuții din modelul dat este direct proporțională cu timpul pe care îl petrece culoarea respectivă pe pixelul referențiat. Astfel, pixelul cu o pondere a distribuțiilor foarte mică este clasificat ca și fundal. Parametrii principali ai acestui *subtractor* sunt după cum urmează:

- **history** - responsabil de numărul de cadre utilizat pentru cumulara de ponderi
- **nmixtures** - indică câte distribuții Gaussiene ar trebui reținute pe întreg parcursul videoclipului sau a setului de cadre
- **backgroundRatio** - definește pragul de diferențiere între fundal și prim plan
- **noiseSigma** - definește nivelul de acceptare a zgomotului. O valoare foarte joasă poate genera obiecte false

Pentru MOG2 (Mixture of Gaussians 2), bazat pe lucrările [9] și [10], a fost întocmit pentru a optimiza limitările MOG1, mai precis, aceste limitări constau în numărul fix de distribuții Gaussiene utilizate.

Utilizând un număr variabil de distribuții Gaussiene, mapate pixel cu pixel, MOG2 reușește să descrie o reprezentare mult mai bună a complexității culorilor din fiecare cadru. Conform testelor făcute de Marcomini, rezultatele sunt prezentate în tabelul 2.2.

Tabelul 2.2. Rezultatele testelor

<i>Categoria</i>	<i>MOG1</i>	<i>MOG2</i>
Acuratețe	89%-98%	93%-99%
Precizie	100%	100%
Timp de procesare	28 cadre/s	64 cadre/s

Testele au fost realizate pe un set de aproximativ 10.000 de cadre. MOG1 a procesat toate cadrele într-un interval de 500 de secunde rezultând 28 de cadre pe secundă, pe când MOG2 a procesat același număr de cadre în 150 de secunde rezultând astfel 64 de cadre pe secundă. În figura 2.2 este prezentat comparația rezultatului în urma aplicării metodelor de extragere a fundalului GMG, MOG1 și MOG2.

MOG2 a fost metoda principală utilizată pentru separarea fundalului de prim-plan, în contextul lucrării prezentate, până când a apărut problema considerării vehiculelor staționate pentru un timp îndelungat ca și fundal eliminându-le prezența din masca finală.

2.1.4. Video stabilization

În lucrarea [11] se vorbește despre metode de stabilizare a filmărilor realizate de către drone automate. Problema este pusă în primul rând la nivelul mișcărilor generate de către deviațiile dronelor chiar dacă ele se află într-o poziție fixă în aer, aceste deviații pot fi cauzate de vibrațiile mecanice ale motoarelor, de bătaia vântului sau de procedurile de auto-fixare a dronelor. Tot în această lucrare se vorbește despre stabilizarea videoclipurilor ca metodă de preprocesare a filmărilor pentru diferitele scopuri intenționate, cum ar fi detecție sau clasificare. La baza stabilizării unei filmări, se află algoritmi de estimare a mișcării. Această estimare a mișcării calculează mișcarea globală între 2 cadre consecutive. Tot în această lucrare se menționează vectorii de mișcare globală, acești vectori pot

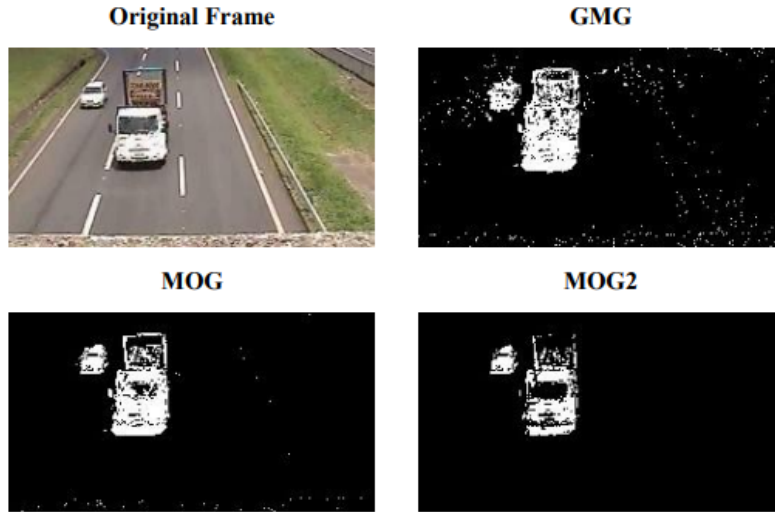


Figura 2.2. Compararea rezultatelor GMG vs MOG1 vs MOG2

prezenta o lipsă de precizie. Așadar, acești vectori ce pot încurca stabilizarea sunt înlocuiți cu metode ce include RANSAC, LMS și MSAC. Principiile matematice din această lucrare sunt aplicate pe modele simple 2D, adică în cazuri unde adâncimea nu prezintă o problemă fundamentală. Modelele de mișcare 2D include translații, rotații plus translații, rotație scalată și modele de mișcare afină și proiectivă. Modelele de translație 2D pot fi exprimate ca în formula (2.5).

$$x' = [I \ t]x \quad (2.5)$$

Unde x este coordonata pixelului în cadrul (t), iar x' sunt coordonatele pixelilor în cadrul ($t+1$). I este matricea unitate.

Translația plus rotația într-un model 2D poate fi exprimată conform formulei (2.5), unde R este matricea de rotație.

$$x' = [R \ t]x \quad (2.6)$$

Modelul de mișcare afină poate fi exprimat conform (2.7).

$$x' = \begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \end{bmatrix} X \quad (2.7)$$

Iar matricea de omografie poate fi exprimată ca în (2.8).

$$x' = \begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \\ a_{20} & a_{21} & a_{22} \end{bmatrix} x \quad (2.8)$$

Stabilizarea înregistrării, în prezenta lucrare, a apărut mult mai târziu față de alte metode deoarece abia când am observat cum masca finală are zone foarte mari unde detecția de fals pozitive apare, am început să îmi pun întrebări. Aparent și micile mișcări care pare ne semnificative pot schimba cum percepe MOG2 fundalul și cum extrag diferențele absolute „diferențele” propriu-zise.

2.1.5. Centroid tracking

În lucrarea [12] principala metodă abordată pentru detecția de mașini dintr-un videoclip filmat cu drona, este o metodă bazată pe detecția de centroizi. Însă, pe lângă această metodă, Shuja Ali prezintă și un set de preprocesări foarte bine documentate. În primul rând, se discută despre eliminarea

zonelor neimportante, adică definirea unei zone de interes, mai precis strict drumul. La acest pas, menționează o metodă foarte interesantă de filtrare a drumului dintr-o filmare, ce permite selectarea unei raze de culori care să includă zona de interes pe detecție și să excludă zonele unde detecția nu va avea loc[13]. Mai departe, sunt prezentate etape intermediare ale preprocesării prin metode de eliminare a zgomotului și netezire a imaginilor utilizând *Gaussian blur* [14], dat fiind un set de date filmat din aer predisus la oscilații și la zgomot. Urmat mai apoi de o conversie către tonuri de gri pentru ca factorul de culoare să nu influențeze detecția. Ultimele etape de preprocesare se rezumă la aplicarea unor operații morfologice cum ar fi dilatarea și eroziunea[15]. În continuare se prezintă detecția și urmărirea vehiculelor utilizând YOLOv3 [16] pentru detecție, iar pentru urmărire utilizează un algoritm de urmărire a centroizilor.

Algoritmul propus de urmărire a vehiculelor utilizând detecția centroizilor, constă în verificarea existenței unui chenar ce încadrează fiecare vehicul în parte, după care se calculează pentru fiecare chenar un centroid respectiv. Pentru partea de urmărire, la fiecare cadru nou, se calculează distanța Euclidiană [17] dintre poziția anterioară a centroidului și noul chenar ce încadrează vehiculul etichetat. Astfel, prin aplicarea unei toleranțe la distanță, se pot aplica identificatori unici pentru fiecare vehicul detectat.

În lucrarea curentă, algoritmul de detecție și urmărire a centroizilor se bazează pe un concept similar, calcularea unui punct central și urmărirea acestuia în timp și spațiu prin niște contori de persistență. Un contor pentru numărul de cadre în care obiectul „așteaptă” să regăsească acel centroid, și un contor pentru distanța parcursă înainte de reatribuirea unui alt identificator unic.

2.2. Algoritmi menționați

2.2.1. Filtrul Kalman

Filtrul Kalman este o metodă recursivă de estimare a stadiului unui sistem dinamic bazat pe o serie de date de intrare ce conțin zgomot. În lucrarea lui Greg Welch[18] este descris atât filtrul Kalman discret și cel extins. Filtrul Kalman discret se adresează unei probleme generale de încercare a estimării unei stări x a unui proces controlat în timp discret dat de ecuația diferențială stohastică 2.9, și măsurătorile date de ecuația 2.10

$$x_{k+1} = A_k X_k + B u_k + w_k, x \in \mathbb{R}^n \quad (2.9)$$

$$z_k = H_k x_k + v_k, x \in \mathbb{R}^m \quad (2.10)$$

. Variabilele aleatoare w_k și v_k reprezintă procesul și zgomotul măsurătorii respectiv. Se presupune a fi independente una de cealaltă și cu distribuția de probabilități normală. Matricea A de dimensiune $n \times n$ în ecuația 2.9 corelează starea la momentul de timp k cu starea la momentul de timp $k + 1$, fie în absența unei funcții de activare sau zgomotul procesului. Matricea B de dimensiunea $n \times 1$ corelează intrarea $u \in \mathbb{R}^1$ cu starea x . Matricea H de dimensiune $m \times n$ din ecuația 2.10 corelează starea k cu măsurătoarea z_k . Bazate pe aceste ecuații, filtrul Kalman estimează un proces folosind un fel de control al *feedback-ului*. Practic filtrul estimează starea procesului la orice moment dat și primește acest *feedback* sub formă de măsurători ce conțin zgomot. Astfel, ecuațiile filtrului Kalman se împart în 2 categorii, ecuațiile ce se bazează pe actualizarea măsurătorilor și cele ce contribuie la actualizarea momentelor. Comparativ cu filtrul Kalman discret, filtrul Kalman extins vine ca soluție la aceeași problemă, însă bazat pe procese neliniare.

Capitolul 3. Soluția propusă

3.1. Problema prezentată

În contextul actual, traficul urban tinde doar să crească, iar infrastructura rutieră din România nu a fost gândită pentru o creștere atât de mare a traficului rutier. Așadar, prin prezenta lucrare se propune un sistem automat de detectare și urmărire a vehiculelor într-un cadru real, utilizând o filmare aeriană. Acest studiu de caz vizează problemele ce apar la nivel local, mai precis intersecția Podu Roș din Iași, un celebru nod rutier cunoscut pentru volumul extrem de ridicat de trafic și un număr foarte mare de puncte de congestie. Problema identificată constă în lipsa instrumentelor automatizate pentru monitorizarea dinamică a fluxului traficului rutier și extragerea de metrice relevante precum densitatea traficului, direcțiile de deplasare cele mai frecventate, identificarea zonelor cu blocaje frecvente și elemente ce îngreunează deplasarea. Obiectivul principal este de a oferi un suport precis spre luarea deciziilor pentru optimizarea traficului în această zonă.

În lucrarea curentă, va fi prezentată o combinație de metode de *computer vision* cu metode de prelucrare a imaginilor video pentru a detecta și urmări vehiculele în timp real, dar și pentru extragerea informațiilor relevante analizei de trafic.

3.2. Tehnologii utilizate

Pentru rezolvarea problemei prezentate, am ales utilizarea OpenCV [19] împreună cu limbajul Python [20].

3.2.1. OpenCV

OpenCV este o librărie *open-source* specializată pentru *computer vision* și prelucrarea imaginilor. Această librărie pune la dispoziție o varietate de algoritmi și unelte pentru realizarea sistemelor de detecție, analiză, recunoaștere și multe altele. Conține peste 2.500 de algoritmi optimizați ce includ detecție facială, identificarea de obiecte, urmărire de obiecte, clasificare de acțiuni umane și extragere de modele 3D. Este conceput pentru analiză în real-time a imaginilor, ceea ce face această librărie mai mult decât potrivită pentru contextul acestei lucrări.

3.3. Operații de pregătire a datelor

În această secțiune vor fi prezentate operațiile aplicate asupra înregistrării pentru a facilita o detecție cât mai precisă și eficientă.

3.3.1. Stabilizarea videoclipului utilizând VideoStab

OpenCV pune la dispoziție și un set de funcții de stabilizare a înregistrărilor prin diferite metode⁵.

Constructorul clasei *VideoStab* (cu echivalentul în Python numit *VidStab*) permite setarea metodei principale de stabilizare prin parametrul *kp_method*. Acest parametru primește un șir de caractere ce reprezintă unul din descriptorii de puncte cheie. Conform ⁶la momentul actual sunt disponibili următorii descriptori: "GFTT", "BRISK", "DENSE", "FAST", "HARRIS", "MSER", "ORB" și "STAR". Pe lângă aceștia ar fi disponibili și "SIFT" și "SURF" însă în funcție de versiunea de OpenCV utilizată poate interveni eroarea de patent. Filmarea fiind realizată cu o dronă, cel mai evident tip de mișcare este cea de rotație. Așadar, descriptorul potrivit pentru stabilizarea mișcărilor de rotație este GFTT.

⁵Video Stabilization OpenCV https://docs.opencv.org/3.4/d5/d50/group__videostab.html

⁶VidStab în Python https://adamspannbauer.github.io/python-video_stab/html/module_docs.html

Din toate metodele puse la dispoziție, personal m-am axat pe 2 dintre ele *OnePassStabilizer* (cu echivalentul în Python intitulat *stabilize_frame*) și *TwoPassStabilizer* (cu echivalentul în Python intitulat *stabilize*). *OnePassStabilizer* este utilizat pentru procesare cadru cu cadru, adică în interiorul unui loop de procesare a videoclipului permițând astfel și alte prelucrări ale cadrelor costul pentru aceasta este precizia și calitatea stabilizării. Metoda *TwoPassStabilizer* utilizează propria buclă de citire și scriere a clipului utilizând funcții OpenCV, nu mai permite prelucrarea suplimentară a cadrelor înainte de scriere, însă oferă o precizie și o calitate mult mai bună față de *OnePassStabilizer*. Ambele metode utilizează parametrul *smoothing_window* care reprezintă dimensiunea ferestrei utilizate în netezirea traiectoriei. Mai exact, reprezintă câte cadre sunt luate în considerare pentru a netezi mișcarea camerei. O valoare mare, poate obține un decalaj între corecție și mișcare, însă asigură un videoclip mai stabil, pe când o valoare mică poate să nu elimine mișcările suficient de mult dar poate aduce o stabilizare mult mai sincronizată la mișcare. Deoarece metoda *OnePassStabilizer* tăia din cadre pe parcursul filmării, cu dimensiunea ferestrei aproape de 150, am ales să folosesc metoda *TwoPassStabilizer* cu dimensiunea ferestrei de 200.

3.3.2. Extragerea fundalului - filtru median

Fundalul filmării a fost extras într-un mod forțat pentru a putea ușura detecția vehiculelor prin metode ce compară filmarea originală cu niște cadre referință unde mașinile nu sunt prezente în aceeași filmare. Dat fiind gradul de aglomerare al intersecției, un cadru nativ fără vehicule prezente în intersecție este o provocare imposibilă. Astfel, am decis să „creez” o întreagă filmare unde vehiculele nu sunt prezente în intersecție. Acest lucru a fost realizat prin utilizarea unui filtru median [1]. La fel ca și Biradar, am utilizat filtrul median pentru a elimina obiectele redundante și pentru a obține un fundal omogen. Însă primele iterații ale utilizării filtrului median au dus la niște rezultate neașteptate. Simpla aplicare a filtrului median a avut efect doar pe anumite zone ale intersecției unde este foarte multă mișcare. După cum este prezentat în figura 3.1, din cauza aglomerării constante a intersecției și a unui timp de staționare foarte mare a vehiculelor în intersecție, filtrul median le-a considerat ca și fundal. Așadar, am venit cu o soluție: crearea unui buffer de cadre peste care să „forțez” filtrul median.



Figura 3.1. Aplicarea filtrului median simplu

În codul funcției *remove_cars*, este prezentat un buffer de tip coadă de cadre, care, atâta timp cât nu are dimensiunea de 360 de cadre, adaugă cadre în coadă până se ajunge la 360. În momentul în care dimensiunea cozii de cadre ajunge la 360 de cadre, matricile respectivelor cadre sunt *stacked*, adică se suprapun a aceste matrici între ele. După care se aplică filtrul median pe aceste cadre, rezultatul obținut devine un nou cadru care este scris la filmarea rezultat. Atributul *.pop(0)* indică eliminarea primului element din coadă, adică primul cadru aflat în coadă, scăzând astfel dimensiunea cozii la 359, adăugând în iterația următoare un nou cadru. Procesul se repetă până când ultimul cadru

din filmare este introdus în buffer. Alegerea numărului de 360 de cadre pentru acest filtru median a fost realizată prin testarea unor dimensiuni și măsurarea timpilor de execuție. În acest context, timpul de execuție al funcției este direct proporțional cu numărul de cadre. Conform figurii 3.2, se poate observa cum un filtru median cu un buffer de 120 nu aduce un rezultat optim, dar asta nu înseamnă că abordarea este greșită, ci pur și simplu, dimensiunea buffer-ului nu era suficientă pentru timpul de staționare realizat de către mașini în intersecție. Pentru această variantă, timpul de procesare a fost în jur de 42 de minute.



Figura 3.2. Aplicarea filtrului median pe un buffer de 120 de cadre

Figura 3.3 prezintă un cadru din filmarea unde a fost aplicat filtrul median cu buffer-ul de 360 de cadre. Iterațiile cu valoarea dimensiunii buffer-ului mai mare rezultau mai multe elemente esențiale șterse (cum ar fi marcăjele din drum), iar o dimensiune mai mică nu reușea să șteargă vehiculele din filmare. Această iterație a funcției a avut un timp de execuție de 2 ore și 34 de minute.



Figura 3.3. Aplicarea filtrului median pe un buffer de 360 de cadre

3.4. Implementarea operațiilor preliminare

3.4.1. Extragerea prim-planului - MOG2

O metodă foarte bună de diferențiere între fundal și prim-plan este utilizarea algoritmului MOG2 (Mixture of Gaussians 2). Este un lucru neobișnuit să utilizez un algoritm de extragere a fundalului chiar dacă fundalul este extras deja, dar din nou, natura specifică a acestei filmări nu ne permite utilizarea algoritmilor în forma lor nativă. În primele teste de utilizare MOG2, algoritmul se prezenta ca fiind impecabil, se separa fundalul de prim-plan, însă, apăreau probleme la staționarea mașinilor pe o perioadă foarte lungă în intersecție, iar dacă vehiculele depășeau numărul de cadre dat ca parametru pentru MOG2 erau considerate fundal și nu mai erau evidențiate în masca rezultat.

Aici intervine filmarea fără vehicule, aplicând MOG2 inițial pe filmarea fără vehicule pentru a evita situațiile în care vehiculele sunt considerate fundal. Chiar și așa algoritmul mai are momente în care vehiculele staționate pentru mult timp sunt catalogate drept fundal.

În figura 3.4 este evidențiată staționarea vehiculelor în pseudo-cadranul 1 al intersecției (filmarea nu este așezată în conformitate cu punctele cardinale așadar voi folosi pseudo-notații trigonometrice fiind în forma de cerc) nu sunt foarte bine evidențiate.

Însă pentru o separare cât mai bună a fundalului de prim-plan, am decis ca MOG2 să facă o „încălzire” înainte de a fi utilizat. În funcția *warmup_mog*, unde parametrul *warmup_frames* care are o valoare inițial setată pe 0, valoare care indică funcției să învețe fundalul de pe toată filmarea dată ca parametru. Orice valoare mai mare de 0 dată pentru acest parametru, va indica numărul de cadre care va fi utilizat în încălzirea lui MOG2. În interiorul codului se poate observa la partea de *background_subtractor.apply(frame, learningRate=-1)* parametrul *learningRate=-1*. Valoarea de -1 nu este dată întâmplător, această valoare indică o rată de învățare adaptivă a fundalului dat de parametrul *frame*. De preferat pentru cazuri generale, parametrul *learningRate* să primească o valoare în intervalul [0, 1]. Evident, o valoare destul de mică pentru acest parametru duce la o rată de învățare mai lentă dar mai precisă, însă, având în discuție filmarea unde vehiculele sunt inexistente, rata de învățare adaptivă produce cele mai bune rezultate. În medie această încălzire aplicată pe tot videoclipul durează aproximativ un minut.

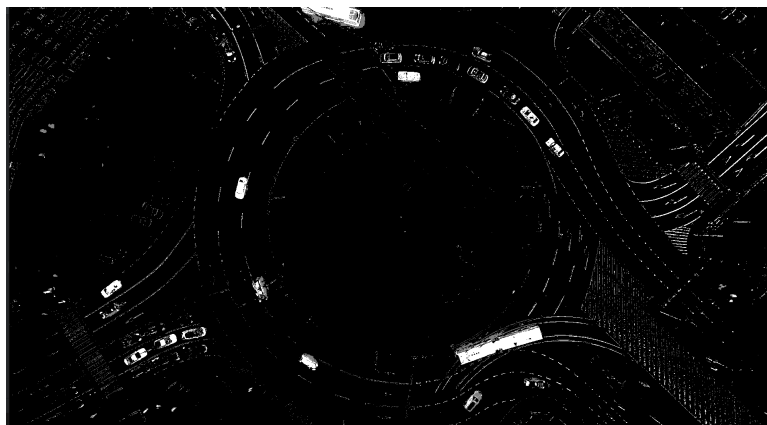


Figura 3.4. Mască rezultat în urma aplicării MOG2

3.4.2. Problema vehiculelor cu nuanțe închise

Vehiculele a căror paletă de culori este foarte închisă se dovedesc a fi o provocare pentru detecție mai ales când cadrele sunt trecute de la *RGB* la *grayscale*. Așadar pentru acest inconvenient, am decis că o mască special construită pentru zonele închise la culoare va fi varianta cea mai bună. Mască din figura 3.5 a fost realizată prin aplicarea funcției OpenCV *cv2.inRange(src, lowerb, upperb)*. Această funcție preia ca parametrii imaginea (*src*) și apoi setează pixelii aflați în intervalul [*lowerb*, *upperb*] pe valoarea 255.

Prin testare continuă, am ajuns la concluzia că cel mai bun interval pentru situația dată, care nu interferează cu alte nuanțe, este [35, 70], un interval mai mare decât acesta setează pe 255 și pixelii ce reprezintă partea carosabilă lucru care îngreunează și mai mult detecția. Chiar și cu această punere în evidență pentru pixelii cu nuanțe închise, se observă în figura 3.5 cum majoritatea vehiculelor ce ies în evidență în această mască au goluri în partea centrală. Pentru aceasta am aplicat o operație morfologică de deschidere și dilatare pentru a umple golurile lăsate de operația *inRange*.

3.4.2.1. Observații

Evidențierea aceasta asupra vehiculelor cu o nuanță închisă duce la mai multe elemente ce produc zgomot în măștile binare. Mare parte din detecțiile false și schimbările bruște de intensități



Figura 3.5. Evidențierea zonelor închise la culoare

apar din cauza acestei măști.

3.4.3. Diferențe absolute

Utilizând filmarea unde au fost șterse vehiculele, se pot utiliza acum ambele în paralel pentru aplicarea de diferențe absolute, cadru cu cadru. Așadar, pentru fiecare cadru din filmarea originala, există un cadru în filmarea prelucrată fără vehicule.

După cum se observa în figura 3.6, algoritmul de diferențe absolute are o toleranță foarte scăzută la evidențierea vehiculelor de culoare închisă cum ar negru sau gri. Însă, diferențele absolute, pot evidenția vehiculele staționate în intersecție.

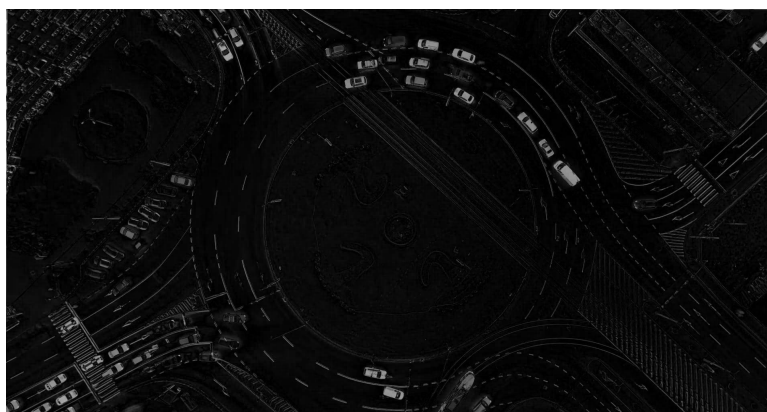


Figura 3.6. Rezultatul aplicării diferențelor absolute

Așadar, prin îmbinarea diferențelor absolute și MOG2 va rezulta o mască ce evidențiază atât vehiculele în mișcare cât și cele staționate. Însă, diferența absolută în acest caz funcționează pe un principiu de atât diferențele pozitive cât și pe cele negative, asta înseamnă că orice diferență dintre filmarea originală și cea unde vehiculele au fost șterse va fi luată în calcul. Așadar un pas intermediar ar fi aplicarea unui *Gaussian blur* [21] de ușoară intensitate va ajuta la un număr crescut de detecții pozitive, problema principală cu această metodă fiind detecțiile false cauzate de prezenta marcajelor rutiere cu o arie foarte mare.

3.4.4. Zona de interes

În lucrul cu nuanțe de gri, mult zgomot se generează din cauza schimbărilor de intensitate luminoasă. Filmarea prezintă o tranziție de la lumină artificială la lumină naturală, ceea ce creează foarte mult zgomot când aducem filmarea într-un stadiu de binarizare. De aceea, pentru a elimina zgomotul generat de reflexia luminii fie în clădiri, fie în panouri, dar și eliminarea marcajelor albe

ce pot interfera cu detecția, am decis definirea unei zone de interes ce să vizeze doar elementele ce includ partea carosabilă. Zona de interes este prezentată în figura 3.7.

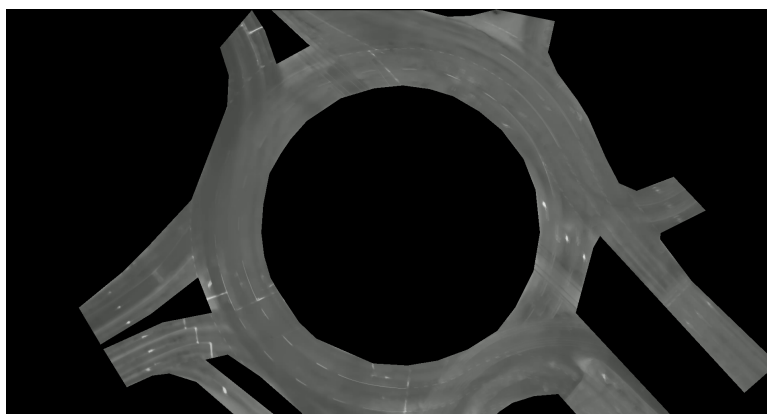


Figura 3.7. Zona de interes definită

În modulul *constants.py* sunt prezentate cele 3 constante utilizate în crearea zonelor de interes, utilizând poligoane complexe. Constanta principală este *included_region* ce definește practic toată zona vizibilă din figura 3.7. Celelalte 2 constante, *excluded_small_white_blob_left* și *excludint_false_detection* sunt definite pentru a exclude anumite porțiuni din zona de interes care sunt sigure că vor genera detecții false, spre exemplu *excluded_small_white_blob_left* definește un marcaj în partea stângă care are o arie suficient de mare cât să fie identificat ca vehicul, iar constanta *excludint_false_detection* definește zona centrală a intersecției care generează foarte multe detecții false în cazurile de schimbări de iluminare. Modul în care a fost creată această mască a fost alegerea coordonatelor prin desenare manuală a zonei de interes. Logica acestei măști este definirea cu alb a zonelor ce vrem să fie incluse, iar cu negru zonele ce urmează a fi excluse, pentru ca la final, în urma unei operații logice *AND* să fie afișată doar o mască cu zona de interes.

3.4.4.1. Observații

Definirea zonei de interes a devenit necesară în urma combinării măștilor principale cu masca ce scoate în prim-plan vehiculele de culoare închisă, deoarece zgomotul lăsat de această mască interferează cu logica de detecție și creea foarte multe zone unde apăreau detecții false și îngreunau, prin imbinarea cu vehiculele din prim-plan, detecția obiectelor ce reprezintă un *true positive*.

3.4.5. Intrările și ieșirile din intersecție

În modulul *constants.py* sunt definite constantele intrărilor și a ieșirilor din intersecție. Pentru intrările în intersecție, am ales utilizarea de poligoane complexe, iar pentru ieșirile din intersecție am ales să utilizez doar segmente. Motivul pentru care am ales definiții diferite ale acestor zone, este timpul de staționare. Pe partea intrărilor, vehiculele staționează un timp relativ mult din diferite motive, cel mai comun fiind statul la semafor sau cedarea de prioritate. Având astfel de poligoane complexe în zonele de intrare, se pot număra mașinile ce urmează să intre în intersecție pe parcurs ce se adună în zona de așteptare. În schimb pentru ieșiri, mașinile nu staționează la părăsirea intersecției, lucru ce m-a determinat să definesc doar niște segmente prin care dacă centrul vehiculului depășește orice punct de pe segment, aceasta mașină este considerată că părăsește intersecția prin drumul specificat. Fiecare dicționar are ca și cheie denumirea străzii pe care o reprezintă.

3.4.6. Operații morfologice

Pentru a continua pașii de pregătire a cadrelor înainte de a aplica pasul de detecție a fost aplicat un set de operații morfologice care să ajute evidențierea caracteristicilor ce facilitează detecția

și eliminarea elementelor nedorite ce împiedică detecția.

3.5. Implementarea logicii de detecție, urmărire și numărare a vehiculelor

3.5.1. Clasa *EnhancedCentroidTracker*

După preprocesarea datelor, se extrage conturul din masca finală reprezentată de figura 3.8 și sunt filtrate ariile de dimensiuni foarte mici. Prin teste am observat că orice arie peste 450 este acceptabilă.

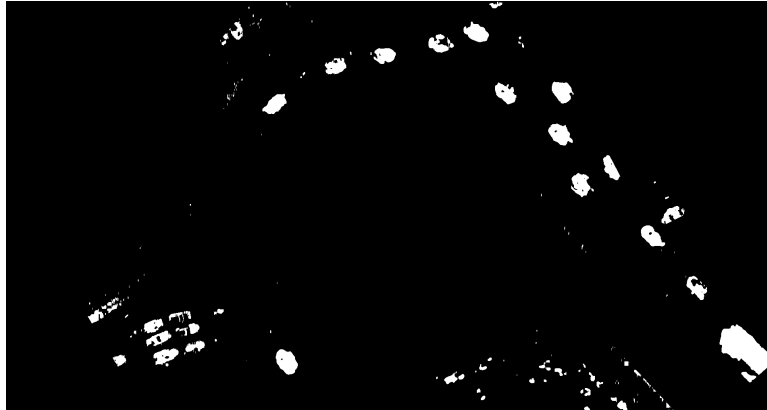


Figura 3.8. Masca finală pe care este realizată detecția

Însă, detecția unui contur nu m-a ajutat să mențin identitatea unică a fiecărei mașini, fără un id unic, nu exista numărare și urmărire. Așadar, am implementat clasa *EnhancedCentroidTracker*, care a fost inspirată de ideea clasică de identificare a centroizilor dintr-o problemă simplă de clasificare. Atributele principale ale acestei clase sunt:

- *objects* - un *OrderedDict* ce stochează pentru fiecare obiect detectat un id unic și informațiile despre poziția sa curentă (maparea de tip cheie -> valoarea pentru acest atribut este *object_id* -> (centroid, bbox)).
- *disappeared* - un dicționar care ține evidența numărului de cadre în care un obiect nu a mai fost detectat, adică și-a pierdut dreptunghiul ce îl încadrează.
- *entry_flags*, *entry_counts*, *exit_counts* - sunt atributele ce aparțin de logica de numărare pentru intrări și ieșiri folosind zonele predefinite.
- *max_disappeared* - reprezintă numărul maxim de cadre în care un obiect poate lipsi de la ultima lui poziție cunoscută până când este considerat „pierdut” definitiv. În cazul în care valoarea acestui parametru nu este ajustată corespunzător, acesta poate duce la pierderea unui obiect cu id unic și reatribuirea unui alt id aceluiași obiect.
- *max_distance* - distanța maximă permisă pentru asocierea unui nou obiect detectat cu un obiect deja existent. De preferat pentru acest atribut sunt valori mari raportate la rezoluția cadrului, o valoare prea mică s-ar putea să atribuie aceluiași vehicul un id diferit de fiecare dată când depășește distanța atribuită.

3.5.1.1. Metoda principală **update**

Metoda principală și cea care deține aproape toată logica acestei clase, este funcția *update*. Parametrul de intrare pentru această funcție este o listă de dreptunghiuri *rects* ce încadrează obiectele detectate în cadru. Fiecare dreptunghi din listă este definit de o tuplă de forma (**x, y, w, h**) unde **x** și **y**

reprezintă coordonatele x și y corespunzătoare punctului cel mai apropiat de originea sistemului, iar w și h reprezintă lățimea respectiv înălțimea dreptunghiului.

Cu această informație se calculează centrul geometric al fiecărui obiect detectat în parte.

După stabilirea centrozilor, se face o verificare a existenței obiectelor în dicționarul de obiecte detectate, dacă nu se află nimic în această listă orice obiect detectat este considerat ca fiind un nou obiect. În cazul alternativ, în care dicționarul nu este gol, se construiește o matrice de distanțe Euclidiene între centrozii existenți și cei nou detectați. După care se asociază obiectele pe baza celei mai mici distanțe, respectând limita impusă de atributul *max_distance*. Urmează partea de actualizare a datelor. În acest set de instrucțiuni, sunt făcute verificări ale obiectelor pentru a vedea dacă un obiect este asociat în mod corect, dacă nu mai este detectat sau dacă depășește valoarea *max_disappeared*. Dacă asocierea este corect realizată, poziția centroidului este actualizată și se resetează contorul de absență al acestuia. Dacă un obiect își pierde dreptunghiul ce îl încadrează, iar ca urmare, acesta nu mai apare ca fiind detectat, contorul de absență crește cu fiecare cadru. Dacă contorul de absență depășește valoare *max_disappeared*, obiectul este considerat pierdut și eliminat din dicționarul de obiecte. Tot în această funcție se realizează și numărarea vehiculelor care intră și ies din intersecție în funcție de drumul pe care îl utilizează la intrarea și respectiv ieșirea din intersecție. Logica din spate, arată că dacă un obiect detectat trece printr-o zonă definită ca și intrare de către constanta *ENTRY_ZONES* și nu a mai fost marcat până acum ca și intrat în intersecție prin această zonă, se înregistrează ca și cum ar fi intrat prin zona menționată. La partea de ieșire din intersecție, este calculată traiectoria unui obiect, iar dacă aceasta se intersectează cu unul dintre segmentele definite ca ieșire, atunci este luat în considerare pentru numărarea mașinilor ce părăsesc intersecția.

3.5.1.2. Utilizarea obiectului de tip *EnhancedCentroidTracker*

Logica de detecție a obiectelor are un set de pași simpli: aglomerațiile de alb sunt luate în considerare dacă aria lor este mai mare de 450, valoare aflată prin experimentare pentru a filtra ariile de dimensiuni foarte mici care au rămas în mască (spre exemplu marcajele rutiere). Pe fiecare cadru sunt detectate seturile de contururi, apoi se află dreptunghiul ce le încadrează pe fiecare în parte, după care sunt adăugate la lista de dreptunghiuri *rects*. La finalul acestui bloc *for*, se apelează metoda *tracker.update(rects)* pentru a actualiza toate dicționarele conform explicațiilor anterioare. Obiectele returnate de funcția *update* sunt afișate pe ecran și numărate. Ca și observație, din punct de vedere vizual, exista posibilitatea ca id-urile să dispară și să reapară fără a-și schimba valoarea, acest fenomen apare din cauză că pipeline-ul de la OpenCV nu este neapărat și rapid.

În figura 3.9 se poate observa cum arată detecția vehiculelor când toate metodele menționate anterior sunt îmbinate.



Figura 3.9. Detecția realizată prin metoda propusă

3.6. Compararea cu un model YOLOv8

3.6.1. Modele pre-antrenate

Pentru a observa performanța și alți indicatori ce pot pune în lumină nivelul de precizie al algoritmului, am decis compararea sa cu un algoritm simplu ce utilizează un model YOLOv8 pre-antrenat. Din păcate modelele pre-antrenate găsite de mine nu au capacitatea necesară de a detecta vehiculele în contextul specific al videoclipului prezentat, motivul principal fiind dat de aria foarte mică a vehiculele raportate la dimensiunea cadrelor. În majoritatea lucrărilor de detecție a vehiculelor utilizând drone automate, dronele în discuție survolează doar intersecții de dimensiuni mici, iar imaginile rezultate din aceste înregistrări aduc aria vehiculelor la o valoare mult mai mare raportată la dimensiunea generală a cadrelor. Așadar, modelele pre-antrenate nu sunt cele mai potrivite pentru a face o comparație în contextul actual.

Date fiind limitările modelelor pre-antrenate, am decis utilizarea unui model antrenat de un set de date specific înregistrării, utilizând YOLOv8.

3.6.2. Descrierea setului de date

Setul de date utilizat în antrenarea modelului YOLO constă în 325 de cadre alese la întâmplare din totalul de cadre pe care îl oferă înregistrarea. Aceste cadre au fost împărțite în 70% date de antrenare, 20% date de validare și 10% date de testare. Pe lângă asta, a fost utilizată o augmentare de la Roboflow⁷ pentru *motion blur* cu intensitate de până la 2 pixeli pentru a realiza ulterior detecția mașinilor în mișcare, rezultând astfel un set final de date de 753 de cadre menținând proporțiile menționate anterior. Rezoluția nativă a acestor cadre îngreuna foarte mult antrenarea modelului așa ca acestea au fost reduse la o dimensiunea de 960x540.

Pentru a testa calitatea setului de date, am adnotat un set de 5 clase (*bus*, *car*, *tram*, *truck*, *van*) pentru a încerca și o clasificare în funcție de tipul vehiculului pentru o extracție de metrici și mai amănunțită.

Ciclul de antrenare al modelului a fost inițializat pentru 200 de epoci, dimensiunea imaginii de 960 (în conformitate cu redimensionarea aplicată la întreg setul de date) și un *batch* de 2 subseturi de date.

⁷Roboflow <https://roboflow.com/>

Capitolul 4. Testarea aplicației și rezultate experimentale

4.1. Rezultatele antrenării modelului

În anexa 11 sunt prezentate rezultatele returnate în urma antrenării modelului. Antrenarea a luat sfârșit după 107 epoci și a salvat cel mai bun model la epoca numărul 7. La această epocă, rezultatele au fost următoarele:

- time - 328.927,
- train/box_loss - 1.96732,
- train/cls_loss - 1.26736,
- train/dfl_loss - 1.24162,
- metrics/precision(B) - 0.50229,
- metric/recall(B) - 0.69143,
- metrics/mAP50(B) - 0.60042,
- metrics/mAP50-95(B) - 0.35708,
- val/box_loss - 1.96614,
- val/cls_loss - 1.17547,
- val/dfl_loss - 1.27425,
- lr/pg0 - 1.27425,
- lr/pg1 - 0.001078,
- lr/pg2 - 0.001078.

4.2. Testarea celor 2 metode

4.2.1. Condițiile de testare

Pentru a evalua ambele metode, vor fi luate în calcul, în primul rând indicatorii de performanță, adică TN (True Positives), FN (False Negatives), FP (False Positive), precision, recall și F1 calculate pe aceeași înregistrare. Pe lângă acestea, va fi calculat și precizia numărării vehiculelor în funcție de intrările și ieșirile pe care le urmează, dar și timpul de procesare petrecut pentru procesarea fiecărui cadru în parte și a întregii înregistrări.

4.2.2. Datele utilizate la măsurarea performanței

Pentru indicatorii de performanță, se va utiliza setul de date de 325 de cadre adnotate la rezoluție nativă fără augmentări. Pentru a evita cazuri marginale, vom evita adnotările făcute pe poligoane complexe filtrând astfel orice are mai mult de 4 coordonate.

Observație: pentru a putea verifica precizia de numărare a ambilor algoritmi, numărul de vehicule ce parcurg intersecția pe parcursul înregistrării a fost numărat în mod manual.

4.3. Rezultate

4.3.1. Indicatorii de performanță

În tabelul 4.1 sunt prezentate diferențele de performanță dintre algoritmul propus și metoda ce utilizează *machine learning*. Evident acest test al performanțelor va fi mereu în favoarea modelului

Indicatorul	Soluția propusă	YOLOv8
TP	9	10259
FP	22	579
FN	44	0
Precision	0.290	0.946
Recall	0.169	1.003
F1	0.214	0.972

Tabelul 4.1. Indicatorii de performanță înregistrați

de învățare automată pentru că este realizat pe un set de cadre adnotat.

4.3.2. Precizia de numărare

Pentru acest test, ambilor algoritmi le-a fost descrisă aceleași zone de interes cu logică de numărare aproape identică. În figurile 4.1 și 4.2 sunt prezentate diferențele de performanțe ale numărării vehiculelor, în funcție de intrări și ieșiri comparate cu valorile reale.

Aceste grafice nu pun în lumină o metodă față de cealaltă, ci mai mult descriu „haosul” ce se află în această intersecție. Câteva mențiuni speciale sunt:

1. Intrarea prin Bd. Socola - algoritmul propus a depășit numărul de vehicule ce au utilizat acest drum ca intrare în intersecție. Această problemă apare din cauză că specific intrarea respectivă are multe marcaje rutiere care au fost eliminate din zona de interes și totuși, din cauza unor mici mișcări ale camerei, acestea apar și fuzionează cu aglomerările de alb deja existente pe mască, creând astfel un nou obiect cu o noua arie și cu un nou id, aducând astfel problema numărării.
2. Intrarea prin Bd. Țuțora - intersecția fiind mare, intrarea prin această zonă nu a putut fi captată cu totul în înregistrare. De aceea, zona de interes de numărare pentru vehicule este foarte mică, pe lângă aceasta, participanții la trafic tind să taie calea autovehiculelor care vin din această zonă, intrând practic în zona de interes și fiind luate în considerare ca și cum ar fi intrat în intersecție venind din această zonă.
3. Intrările dinspre Nicolina și Str. Palat - din nou, aici apar discrepanțe foarte mari, iar problema aici sunt din nou marcajele rutiere albe care au fost special excluse pentru a nu crea confuzie, dar și felul în care cade lumina pe aceste două intrări. Sunt multe momente pe parcursul înregistrării când lumina bate mai tare pe aceste două străzi, crescând astfel intensitatea pixelilor dintre vehicule, făcând din 3 aglomerări de alb, un singură.
4. Ieșirea spre Șoseaua Națională - acest drum conectat la intersecție este doar pe graficul de ieșire din intersecție fiind un sens unic. Din nou apare o valoare mai mare decât cea reală la numărare deoarece mulți dintre șoferii care depășesc pragul pentru a fii numărați spre acea ieșire, doar staționează undeva aproape. După un timp, lucru care se repetă cu multe vehicule, șoferii dau cu spatele și ies dinspre Șoseaua Națională chiar dacă este sens unic spre ieșirea din intersecție. Un id unic este înregistrat o singură dată, dar din moment ce același autovehicul dispăre din cadru și se reîntoarce acesta va fi numărat din nou.

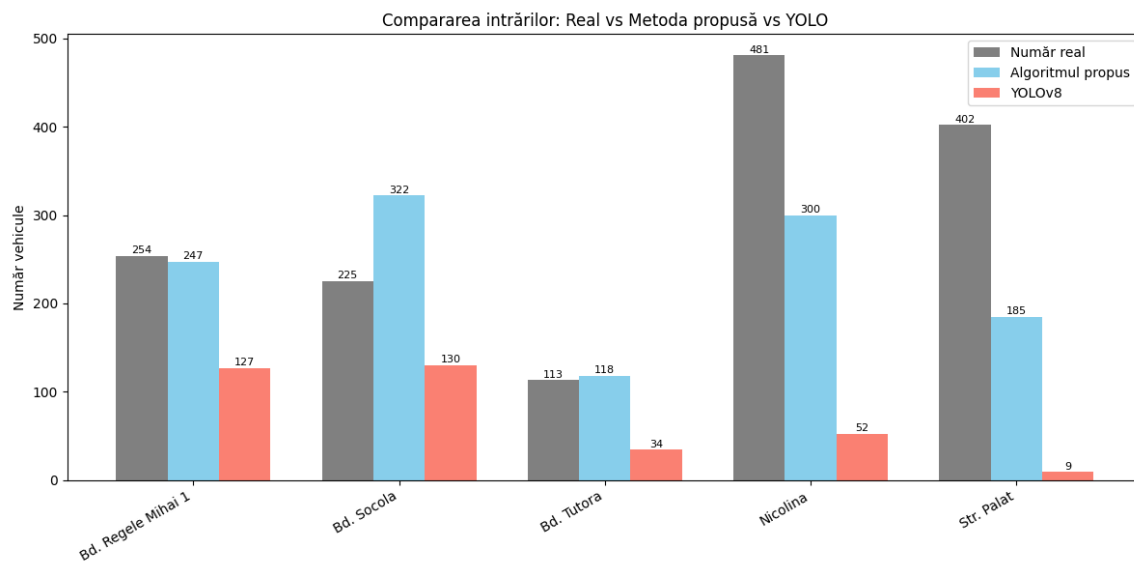


Figura 4.1. Comparare între rezultatele numărării la intrări

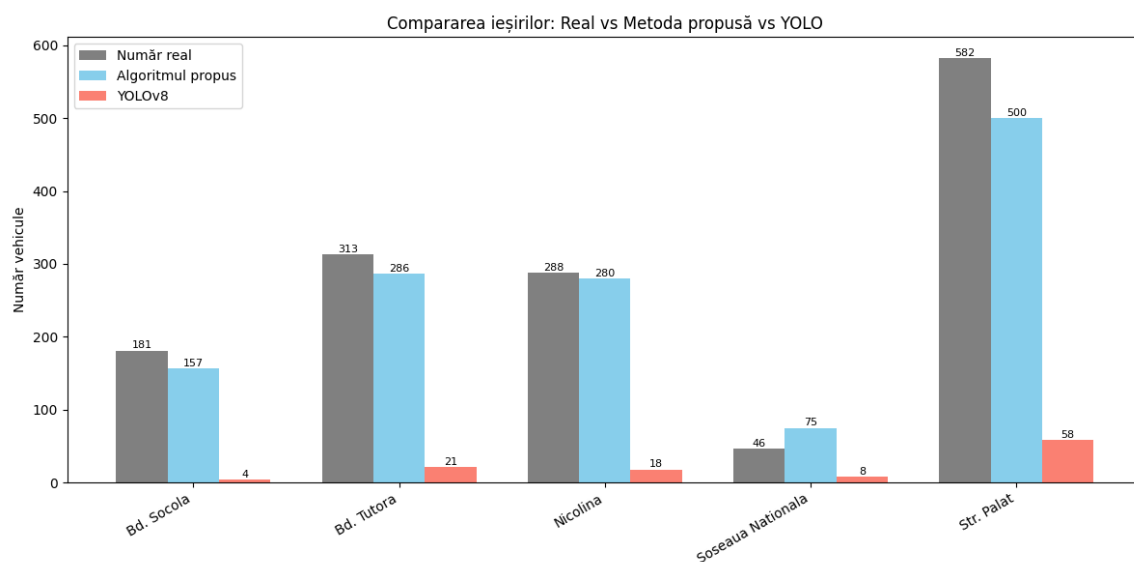


Figura 4.2. Comparare între rezultatele numărării la ieșiri

4.3.3. Măsurarea timpilor de procesare

Un alt test de comparare a celor două metode este timpul de procesare total pentru toată înregistrarea dar și pentru fiecare cadru individual. Soluția propusă reușește să proceseze toată filmarea cu tot cu scrierea pe disk în 5 minute și 31 de secunde și cu o medie a timpului de execuție de 0.137 secunde per cadru. În schimb, această metodă este dezavantajată de timpul mediu de „încălzire” pentru MOG2 de 1 minut și 33 de secunde. Metoda ce utilizează *machine learning* are un timp de execuție total cu tot cu scrierea pe disk de 9 minute și 40 de secunde, cu un timp mediu de procesare per cadre individuale de 0.164 secunde per cadru. În figura 4.3 este prezentat timpul de procesare pentru ambele metode comparate pe fiecare cadru în parte.

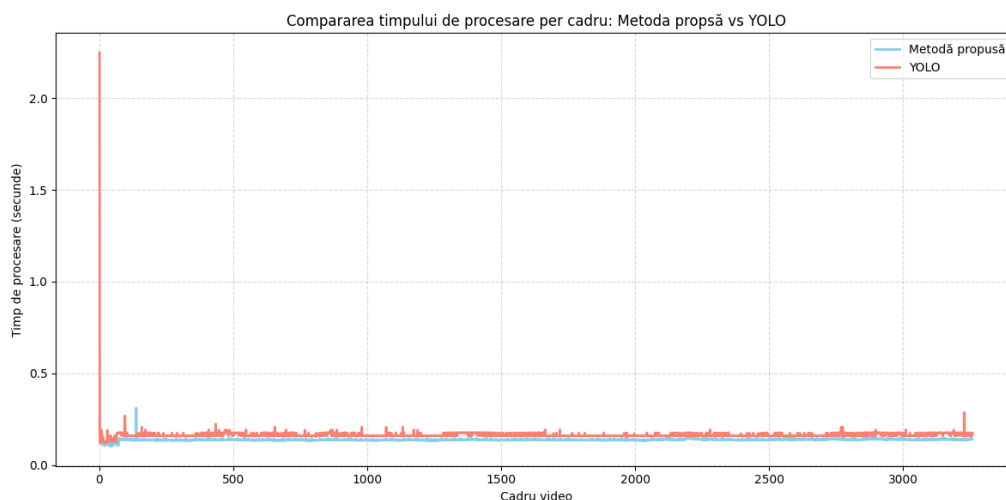


Figura 4.3. Compararea timpului de procesare pe fiecare cadru

Primul cadru dezavantajează foarte mult modelul YOLOv8, atingând aproape 3 secunde timp de procesare, însă începând cu al doilea cadru timpul de procesare scade și ajunge chiar să concureze cu viteza de procesare per cadru a algoritmului propus. Diferența în timpii de procesare per cadru apare și prin diferența de metodă, algoritmul propus face direct prelucrări pe o mască binară cu niște calcule de bază, pe când modelul YOLO se bazează pe predicții a unde se află vehicule bazat pe datele de la antrenare.

4.3.4. Opinii personale asupra rezultatelor

Din punct de vedere al detecțiilor, pot spune cu certitudine că algoritmul ce utilizează *machine learning* este mult mai precis. Marele avantaj al acestor algoritmi ce utilizează învățarea automată este generalizarea lor, o dată ce un algoritm din această categorie este antrenat cu seturi noi de date și seturi de date tot mai mari și tot mai generale, algoritmul devine mult mai performant în contexte generaliste. Însă, dacă ne uităm din punct de vedere local, mai precis la contextul înregistrării analizate, vom observa cum apar multe probleme din punct de vedere al setului de date pe care sunt testați algoritmi. Rezultatele arată prin indicatorii de performanță a detecției pun în lumină metoda ce utilizează *machine learning* ca fiind mult mai precisă. Însă, soluția propusă este o metodă dezvoltată și personalizată pentru contextul acestei probleme locale, lucru care se poate observa și în rezultatele anterioare. Chiar și cu dezvoltarea aceasta specifică, setul de date se dovedește a fi o piedică în apropierea de un rezultat cât mai apropiat de realitate. Un lucru pe care l-am observat este și modul de abordare a intersecției din punctul de vedere al șoferilor. Încadrările neadecvate ale conducătorilor auto duc deseori la detectarea unui singur *cluster* în loc de două, rezultând astfel un vehicul în minus sau o numărare în plus. Un alt factor ce a influențat numărul de vehicule în funcție de drumurile pe care le parcurg sunt șoferii parcați în zone foarte ciudate, care o dată ce își încep călătoria fac manevre și mai ciudate. Spre exemplu, o mare parte din detecțiile vehiculelor ce părăsesc intersecția spre

Șoseaua Națională, acest drum având sens unic, nu se poate intra în intersecție de aici, staționează la intrarea pe acest drum, după care se întorc înapoi în intersecție, ceea ce duce la un număr multiplu de înregistrări pentru că părăsesc zona de interes, iar atunci când se întorc le este atribuit un id nou.

Capitolul 5. Concluzii

5.1. Gradul de realizare al temei propuse

Din propriul punct de vedere această temă putea fi realizată mult mai bine față de cum se află în stadiul actual. Însă, faptul că am putut să experimentez și să îmbin diferite metode care au dus într-un final la confirmarea ipotezei mele: intersecția în discuție nu doar că nu este proiectată corespunzător pentru utilizarea eficientă a participanților la trafic, dar orice încercare de monitorizare poate fi aproape imposibilă. Comparând această lucrare, care se încadrează la categoria de detecție, urmărire și numărare a vehiculelor din intersecții filmate cu drone, cu alte lucrări similare, această intersecție este extrem de complicată. În primul rând, dimensiunea intersecției este mult peste limitele detecțiilor fie ele metode clasice, fie ele metoda cu *machine learning* sau o combinație dintre cele două. Volumul intersecției face ca în orice înregistrare vehiculele să apară de dimensiuni foarte reduse, lucru care îngreunează foarte mult detecția. În al doilea rând avem viteza de redare a videoclipului. Fiind o viteză de redare mult prea mare de redare, se induce un efect de deplasare prea bruscă a vehiculelor, ajungând ca unele vehicule să se deplaseze distanțe relativ mari de la un cadru la altul. În al treilea rând avem o zonă de interes foarte mică comparativ cu zonele ce pot produce zgomot. Zona care, puteai face parte din infrastructura rutieră, creând astfel mai mult spațiu pe partea carosabilă pentru participanții la trafic, dar totodată oferind mai multe opțiuni de a ajunge la destinație. Și nu în ultimul rând avem ca factor de influență al detecției și educația rutieră. Infrastructura rutieră nu este nici ea de cea mai înaltă calitate, însă nu ne putem mândri nici cu educație rutieră a majorității șoferilor români. Din cauza unor încadrări ciudate și fără sens s-a ajuns de foarte multe ori la numărări duble sau poate chiar triple ce au dus la extragerea de metrice eronate.

5.2. Dificultăți întâmpinate

5.2.1. Lipsa cadrelor „goale”

Prima și una dintre cele mai importante dificultăți din această lucrare a fost lipsa unui cadru care să nu aibă vizibile vehicule în intersecție. Prin urmare am fost nevoit să caut metode de eliminare a vehiculelor din imagini pentru a putea avea un set de cadre referință cu care să pot compara cadrele originale. Cum a fost explicat și anterior la capitolul 3.3.2 utilizarea filtrului median a rezolvat această problemă.

5.2.2. Înregistrarea instabilă

Cu mult după ce a început dezvoltarea algoritmului de detecție am observat cum MOG2 și diferențele absolute eșuează la eliminarea elementelor fundal din cauza mișcărilor de rotație a camerei, lucru care o dată la câteva sute de cadre genera foarte mult zgomot și aducea în prim plan și mai multe obiecte ce nu ar fi trebuit detectate. Așadar, după cum am explicat și în capitolul 3.3.1, aplicarea metodei de stabilizare utilizând *VideoStab* a adus înregistrarea într-o formă stabilă cu care a devenit mult mai ușor să lucrez.

5.3. Comparația cu alte proiecte similare

O lucrare care m-a inspirat să antrenez un model de învățare automată este lucrarea lui Sergio[22], unde conceptul este aproape similar însă el nu folosește neapărat o dronă ci creează un algoritm general ce utilizează *machine learning* pentru a detecta vehiculele din diferite unghiuri, pe lângă asta le mai și clasifică în funcție de model și de culoare. Diferența cea mai mare pe care o putem observa între lucrarea mea și lucrarea lui Sergio este setul de date pe care lucrăm. Setul de date utilizat de către Sergio pentru antrenare a fost un set de date generalist cu vehicule clare, care raportate la dimensiunea cadrului au o arie foarte mare comparativ cu ariile pe care le-am obținut din prelucrările

mele. Spre exemplu cea mai mică arie pe care o pot considera vehicul este de 500 care aceasta poate fi aproximată la un dreptunghi de 25 x 20. Raportând această dimensiune la rezoluția de 1920 x 1080 acest vehicul este minuscul. În schimb, în lucrarea lui Sergio, algoritmul fiind generalizat cu cadre unde vehiculele sunt perfect clare, poate fi utilizat pentru camere de bord sau camere ce veghează asupra intersecțiilor dintr-un punct terestru (spre exemplu: stâlpi de iluminare, semafoare sau radare fixe).

5.4. Posibile direcții de dezvoltare

O primă direcție viitoare ar fi implementarea unui filtru Kalman pentru a face predicții ale traiectoriilor centroizilor aflați în mișcare pentru a avea posibilități și mai multe de a extrage date statistice astfel de seturi de date.

O altă posibilă direcție de dezvoltare ar fi generalizarea algoritmului, fie aceasta prin metode clasice, metode cu învățare automată sau un hibrid între cele două. Simt că în România avem nevoie de sisteme automate de monitorizare a traficului rutier din cauza infrastructurii precare și a lipsei de educație rutiere în rândul participanților la trafic. Motivul pentru care am avea nevoie de o astfel de aplicație nu ar fi doar pentru găsirea și sancționarea celor vinovați de diferite accidente rutiere, dar și pentru identificarea zonelor problematice a traficului și a infrastructurii rutiere.

Bibliografie

- [1] R. L. Biradar and A. V. V. Kohir, "Object removal using recursive median filter," *International Journal of Electronics, Computer and Communications Technologies*, vol. 3, no. 4, pp. 14–22, 2013.
- [2] M. Bertalmio, G. Sapiro, V. Caselles, and C. Ballester, "Image inpainting," in *Proceedings of the 27th annual conference on Computer graphics and interactive techniques*, 2000, pp. 417–424.
- [3] S. Roth and M. J. Black, "Fields of experts: A framework for learning image priors," in *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05)*, vol. 2. IEEE, 2005, pp. 860–867.
- [4] R. L. Biradar and V. V. Kohir, "Texture inpainting using covariance in wavelet domain," *Journal of Intelligent Systems*, vol. 22, no. 3, pp. 299–315, 2013.
- [5] K. Youssef and W. Peng-Yung, "Difference of the absolute differences-a new method for motion detection," *International Journal of Intelligent Systems and Applications*, vol. 4, no. 9, p. 1, 2012.
- [6] L. A. Marcomini and A. L. Cunha, "A comparison between background modelling methods for vehicle segmentation in highway traffic videos," *arXiv preprint arXiv:1810.02835*, 2018.
- [7] P. KaewTraKulPong and R. Bowden, "An improved adaptive background mixture model for real-time tracking with shadow detection," *Video-based surveillance systems: Computer vision and distributed processing*, pp. 135–144, 2002.
- [8] C. Stauffer and W. E. L. Grimson, "Adaptive background mixture models for real-time tracking," in *Proceedings. 1999 IEEE computer society conference on computer vision and pattern recognition (Cat. No PR00149)*, vol. 2. IEEE, 1999, pp. 246–252.
- [9] Z. Zivkovic, "Improved adaptive gaussian mixture model for background subtraction," in *Proceedings of the 17th International Conference on Pattern Recognition, 2004. ICPR 2004.*, vol. 2. IEEE, 2004, pp. 28–31.
- [10] Z. Zivkovic and F. Van Der Heijden, "Efficient adaptive density estimation per image pixel for the task of background subtraction," *Pattern recognition letters*, vol. 27, no. 7, pp. 773–780, 2006.
- [11] A. Yousaf, K. Khurshid, M. J. Khan, and M. S. Hanif, "Real time video stabilization methods in ir domain for uavs—a review," in *2017 Fifth International Conference on Aerospace Science & Engineering (ICASE)*. IEEE, 2017, pp. 1–9.
- [12] S. Ali and J. Ahmad, "Vehicle detection and tracking from aerial imagery via yolo and centroid tracking," in *ICACS*, 2023.
- [13] A. Jalal, M. Batool, and K. Kim, "Sustainable wearable system: Human behavior modeling for life-logging activities using k-ary tree hashing classifier," *Sustainability*, vol. 12, no. 24, p. 10324, 2020.
- [14] A. M. Qureshi, A. H. Butt, and A. Jalal, "Highway traffic surveillance over uav dataset via blob detection and histogram of gradient," in *2023 4th international conference on advancements in computational sciences (ICACS)*. IEEE, 2023, pp. 1–5.

- [15] M. Pervaiz, A. Jalal, and K. Kim, “Hybrid algorithm for multi people counting and tracking for smart surveillance,” in *2021 International Bhurban conference on applied sciences and technologies (IBCAST)*. IEEE, 2021, pp. 530–535.
- [16] A. A. Rafique, A. Jalal, and K. Kim, “Automated sustainable multi-object segmentation and recognition via modified sampling consensus and kernel sliding perceptron,” *Symmetry*, vol. 12, no. 11, p. 1928, 2020.
- [17] G. Amato, L. Ciampi, F. Falchi, and C. Gennaro, “Counting vehicles with deep learning in onboard uav imagery,” in *2019 IEEE Symposium on Computers and Communications (ISCC)*. IEEE, 2019, pp. 1–6.
- [18] G. Welch, G. Bishop *et al.*, “An introduction to the kalman filter,” 1995.
- [19] G. Bradski, A. Kaehler *et al.*, “Opencv,” *Dr. Dobb’s journal of software tools*, vol. 3, no. 2, 2000.
- [20] W. Python, “Python,” *Python releases for windows*, vol. 24, 2021.
- [21] E. S. Gedraite and M. Hadad, “Investigation on the effect of a gaussian blur in image filtering and segmentation,” in *Proceedings ELMAR-2011*. IEEE, 2011, pp. 393–396.
- [22] S. S. Sánchez, “Vehicledetectiontracker: Real-time vehicle detection and tracking with yolo,” https://github.com/sergio11/vehicle_detection_tracker, 2023, Ultima accesare: 25.06.2025.

Anexe

Anexa 1. Codul funcției `remove_cars`

```

1  def remove_cars(video_capture):
2      # Definirea unui buffer pentru cadrele necesare
3      buffer = []
4      # Main loop
5      while video_capture.isOpen():
6          # Se citește fiecare cadru
7          ret, frame = video_capture.read()
8          if not ret:
9              break
10         # Pentru un buffer cu mai puțin de 360 de cadre
11         # se adaugă noi cadre
12         if len(buffer) < 360:
13             buffer.append(frame)
14         # Când se ajunge la 360 de cadre se aplică filtrul median
15         # se elimină primul cadru din coadă și se scrie rezultatul
16         if len(buffer) == 360:
17             median_frame = np.median(np.stack(buffer,
18                 ↪ axis=0)).astype(np.uint8)
19             out.write(median_frame)
20             buffer.pop(0)

```

Listing 1. Codul funcției `remove_cars`

Anexa 2. Codul funcției `warmup_mog`

```

1  def warmup_mog(video, background_subtractor, warmup_frames=0):
2      """
3      :param video: Video citit cu funcția cv.VideoCapture
4      :param background_subtractor: Background subtractor inițializat cu
5      ↪ cv.BackgroundSubtractorMOG2
6      :param warmup_frames: Valoare default 0 care indică „învățarea”
7      ↪ fundalului pe parcursul întregului video, orice valoare mai mare de
8      ↪ 0 reprezintă numărul de cadre din care să învețe fundalul
9      """
10     video.set(cv.CAP_PROP_POS_FRAMES, 0) # Pentru siguranță, plecăm de la
11     ↪ cadrul 0
12     if warmup_frames > 0:
13         for _ in range(warmup_frames):
14             ret, frame = video.read()
15             if not ret:
16                 break
17             background_subtractor.apply(warm_frame, learningRate=-1) # -1
18             ↪ înseamnă ca rata de învățare se adaptează
19     else:
20         while video.isOpened():
21             ret, frame = video.read()
22             if not ret:
23                 break

```

```

19         background_subtractor.apply(frame, learningRate=-1)
20     return background_subtractor

```

Listing 2. Codul funcției warmup_mog

Anexa 3. Codul constantelor pentru zona de interes

```

1 included_region = np.array([ [301, 941], [245, 850], [182, 744], [291,
    ↪ 655], [355, 591], [462, 476], [551, 243], [555, 166], [533, 97],
    ↪ [612, 5], [644, 55], [673, 139], [757, 93], [662, 5], [1037, 5],
    ↪ [1099, 21], [1155, 19], [1220, 3], [1365, 32], [1348, 109], [1371,
    ↪ 160], [1422, 229], [1448, 271], [1492, 372], [1525, 435], [1569,
    ↪ 454], [1661, 419], [1740, 502], [1628, 558], [1627, 574], [1705,
    ↪ 677], [1919, 883], [1837, 955], [1478, 565], [1461, 594], [1417,
    ↪ 757], [1709, 1070], [1477, 1073], [1374, 958], [1314, 963], [1260,
    ↪ 980], [1227, 1002], [1161, 1070], [998, 1074], [999, 1048], [886,
    ↪ 1044], [882, 1070], [777, 1062], [736, 1005], [632, 914], [552,
    ↪ 860], [495, 873], [743, 1070], [641, 1072], [474, 928], [407, 907],
    ↪ [365, 912], [301, 941] ], dtype=np.int32)s
2
3 excluded_small_white_blob_left = np.array([ [512, 756], [478,
    ↪ 669], [361, 760], [227, 841], [235, 850], [390, 780], [512, 756] ],
    ↪ dtype=np.int32)
4
5 excluding_false_detection = np.array([ [678, 702], [659, 670], [643,
    ↪ 615], [636, 557], [641, 487], [656, 411], [684, 362], [709, 325],
    ↪ [749, 280], [833, 224], [917, 199], [988, 192], [1054, 201], [1113,
    ↪ 219], [1178, 257], [1233, 303], [1287, 374], [1317, 451], [1328,
    ↪ 557], [1312, 653], [1266, 744], [1222, 801], [1149, 847], [1073,
    ↪ 875], [999, 887], [914, 882], [772, 820], [730, 780], [678, 702]
    ↪ ], dtype=np.int32)

```

Listing 3. Constantele utilizate în definirea zonei de interes

Anexa 4. Codul constantelor utilizate pentru definirea intrărilor și ieșirilor din intersecție

```

1 ENTRY_ZONES = {
2     "Nicolina" : np.array([[551, 232], [557, 175], [532, 86],
3         ↪ [640, 52], [671, 126], [551, 232]]),
4
5     "Bd. Socola": np.array([[307, 942], [259, 849], [436, 760],
6         ↪ [452, 793], [473, 795], [479, 828],
7         ↪ [512, 833], [515, 871], [307, 942]]),
8
9     "Bd. Tutora": np.array([[988, 1072], [1318, 957], [1227, 1077],
10        ↪ [988, 1072]]),
11
12     "Str. Palat": np.array([[1644, 755], [1727, 678], [1768, 734],
13        ↪ [1969, 800], [1644, 755]]),
14
15     "Bd. Regele Mihai 1": np.array([[1542, 437], [1666, 419],
16        ↪ [1737, 504], [1644, 574],
17        ↪ [1542, 437]])

```

```

17 }
18
19 EXIT_ZONES = {
20     "Nicolina" : ((744, 87), (1053, 3)),
21
22     "Bd. Socola": ((387, 558), (458, 711)),
23
24     "Bd. Tutora": ((408, 918), (912, 1052)),
25
26     "Str. Palat": ((1418, 1021), (1551, 882)),
27
28     "Soseaua Nationala": ((1162, 17), (1367, 131))
29 }

```

Listing 4. Constantele utilizate în definirea intrărilor și ieșirilor din intersecție

Anexa 5. Codul utilizat pentru crearea măștii zonei de interes

```

1  # Extragerea dimensiunilor imaginii originale
2  h, w = background_gray.shape[:2]
3  # Definirea unei imagini identice cu toți pixelii setați pe 0
4  roi = np.zeros((h, w), dtype=np.uint8)
5  # Umplerea zonei dorite cu alb, iar zona exclusă cu negru
6  cv.fillPoly(roi, [included_region], (255, 255, 255))
7  cv.fillPoly(roi, [excluded_small_white_blob_left], (0, 0, 0))
8  cv.fillPoly(roi, [excluding_false_detection], (0,0,0))

```

Listing 5. Logica pentru crearea zonei de interes

Anexa 6. Codul de preprocesare a cadrelor

```

1  # Conversia la nuanțe de gri
2  frame_gray = cv.cvtColor(frame, cv.COLOR_BGR2GRAY)
3  # Aplicarea MOG2 împreună cu un median blur pentru eliminarea
   → zgomotului
4  fg_mask = mog2.apply(frame)
5
6  fg_mask = cv.medianBlur(fg_mask, 7)
7  _, fg_mask = cv.threshold(fg_mask, 127, 255, cv.THRESH_BINARY)
8  # Scoaterea în evidență a vehiculelor de nuanțe închise
9  dark_mask = cv.inRange(frame_gray, 35, 70)
10 dark_mask = cv.morphologyEx(dark_mask, cv.MORPH_OPEN, (7,7))
11 dark_mask = cv.dilate(dark_mask, (5,5), iterations=2)
12 # Aplicarea diferențelor absolute între cadrul prim-plan și cel fundal
   → pentru detecția mașinilor staționate
13 blurred_frame = cv.GaussianBlur(frame_gray, (5,5), 0)
14 blurred_bg = cv.GaussianBlur(background_gray, (5,5), 0)
15 diff = cv.absdiff(blurred_frame, blurred_bg)
16 _, diff_mask = cv.threshold(diff, 80, 255, cv.THRESH_BINARY)
17
18 # Combinarea tuturor măștilor
19 combined_mask = cv.bitwise_or(fg_mask, diff_mask)
20 combined_mask = cv.bitwise_or(dark_mask, combined_mask)

```

```

21
22 # Operații morfologice utilizate pentru ajustări în funcție de nevoi
23 combined_mask = cv.morphologyEx(combined_mask, cv.MORPH_CLOSE, (5,5),
    ↪ iterations=2)
24 combined_mask = cv.morphologyEx(combined_mask, cv.MORPH_OPEN, (3,3),
    ↪ iterations=2)
25 combined_mask = cv.dilate(combined_mask, (3,3), iterations=2)
26 combined_mask = cv.erode(combined_mask, (7,7), iterations=1)
27 # Aplicarea zonei de interes
28 combined_mask = cv.bitwise_and(combined_mask, combined_mask, mask=roi)

```

Listing 6. Logica de preprocesare

Anexa 7. Codul clasei EnhancedCentroidTracker

```

1 class EnhancedCentroidTracker:
2     def __init__(self, max_disappeared=50, max_distance=100):
3         """
4         Constructor
5         :param max_disappeared: Numărul maxim de cadre pe care îl
        ↪ „așteaptă” clasa pentru a actualiza id-ul centroidului
6         :param max_distance: Distanța maximă pe care se menține același id
7         """
8         self.next_object_id = 0
9         self.objects = OrderedDict() # dicționar: object_id ->
        ↪ (centroid, bbox)
10        self.disappeared = OrderedDict() # object_id -> frame count
11        self.max_disappeared = max_disappeared
12        self.max_distance = max_distance
13
14        # Atribute pentru numărarea mașinilor în funcție de intrare și
        ↪ ieșire
15        self.entry_flags = {} # object_id -> zone_name - numele străzii
        ↪ de pe care se intră
16        self.previous_centroids = {} # object_id -> centroidul anterior
17        # ENTRY_ZONES și EXIT_ZONES sunt niște constante definite în
        ↪ fișierul constants.py, reprezintă coordonatele date pentru
        ↪ formarea zonelor de interes în care vor fi numărate
        ↪ mașinile, fiecare constantă va apărea în următoarele anexe
18        self.entry_counts = {zone: 0 for zone in ENTRY_ZONES}
19        self.exit_counts = {line: 0 for line in EXIT_ZONES}
20
21
22    def register(self, centroid, bbox):
23        """
24        :param centroid: Centroidul ce reprezintă obiectul identificat
25        :param bbox: Patrulaterul ce încadrează obiectul reprezentat de
        ↪ centroid
26        """
27        self.objects[self.next_object_id] = (centroid, bbox)
28        self.disappeared[self.next_object_id] = 0
29        self.previous_centroids[self.next_object_id] = centroid
30
31        self.next_object_id += 1

```

```

32
33     def deregister(self, object_id):
34         """
35         Funcția ce șterge obiectul specificat de object_id
36         """
37         del self.objects[object_id]
38         del self.disappeared[object_id]
39         if object_id in self.entry_flags:
40             del self.entry_flags[object_id]
41         if object_id in self.previous_centroids:
42             del self.previous_centroids[object_id]
43
44     def update(self, rects):
45         """
46         :param rects: Lista de dreptunghiuri corespunzătoare obiectelor
47         ↪ detectate într-un cadru
48         """
49         if len(rects) == 0:
50             for object_id in list(self.disappeared.keys()):
51                 self.disappeared[object_id] += 1
52                 if self.disappeared[object_id] > self.max_disappeared:
53                     self.deregister(object_id)
54             return self.objects
55
56         input_centroids = np.zeros((len(rects), 2), dtype="int")
57         input_bboxes = []
58
59         for (i, (x, y, w, h)) in enumerate(rects):
60             cx = int(x + w / 2)
61             cy = int(y + h / 2)
62             input_centroids[i] = (cx, cy)
63             input_bboxes.append((x, y, w, h))
64
65         if len(self.objects) == 0:
66             for i in range(len(input_centroids)):
67                 self.register(input_centroids[i], input_bboxes[i])
68         else:
69             object_ids = list(self.objects.keys())
70             object_data = list(self.objects.values())
71             object_centroids = [data[0] for data in object_data]
72
73             D = np.linalg.norm(np.array(object_centroids)[: ,
74                 ↪ np.newaxis] - input_centroids, axis=2)
75
76             rows = D.min(axis=1).argsort()
77             cols = D.argmin(axis=1)[rows]
78
79             used_rows = set()
80             used_cols = set()
81
82             for (row, col) in zip(rows, cols):
83                 if row in used_rows or col in used_cols:
84                     continue

```

```
83         if D[row][col] > self.max_distance:
84             continue
85
86         object_id = object_ids[row]
87         old_centroid = self.objects[object_id][0]
88         new_centroid = input_centroids[col]
89
90         self.previous_centroids[object_id] = old_centroid
91         self.objects[object_id] = (new_centroid,
92             ↪ input_bboxes[col])
93         self.disappeared[object_id] = 0
94
95         # Entry zone detections
96         if object_id not in self.entry_flags:
97             for zone_name, polygon in ENTRY_ZONES.items():
98                 if cv.pointPolygonTest(polygon,
99                     ↪ (int(old_centroid[0]),
100                     ↪ int(old_centroid[1])), False) >= 0:
101                     self.entry_flags[object_id] = zone_name
102                     self.entry_counts[zone_name] += 1
103                     break
104
105         # Exit point line crossing
106         if object_id in self.previous_centroids:
107             p_old = self.previous_centroids[object_id]
108             p_new = new_centroid
109
110             for line_name, (pt1, pt2) in EXIT_ZONES.items():
111                 if self._line_crossed(p_old, p_new, pt1, pt2):
112                     self.exit_counts[line_name] += 1
113
114         used_rows.add(row)
115         used_cols.add(col)
116
117         unused_rows = set(range(0,
118             ↪ D.shape[0])).difference(used_rows)
119         unused_cols = set(range(0,
120             ↪ D.shape[1])).difference(used_cols)
121
122         for row in unused_rows:
123             object_id = object_ids[row]
124             self.disappeared[object_id] += 1
125             if self.disappeared[object_id] > self.max_disappeared:
126                 self.deregister(object_id)
127
128         for col in unused_cols:
129             self.register(input_centroids[col], input_bboxes[col])
130
131         return self.objects
132
133 def _ccw(self, A, B, C):
134     return (C[1] - A[1]) * (B[0] - A[0]) > (B[1] - A[1]) * (C[0] -
135     ↪ A[0])
```

```

130
131 def _line_crossed(self, p1, p2, l1, l2):
132     return self._ccw(p1, l1, l2) != self._ccw(p2, l1, l2) and
        ↪ self._ccw(p1, p2, l1) != self._ccw(p1, p2, l2)

```

Listing 7. Codul clasei EnhancedCentroidTracker

Anexa 8. Secțiunea de cod pentru detecție

```

1  rects = []
2  for cnt in contours:
3      area = cv.contourArea(cnt)
4      if area > 450:
5          x, y, w, h = cv.boundingRect(cnt)
6          rects.append((x, y, w, h))
7
8  objects = tracker.append((x, y, w, h))

```

Listing 8. Logica de detecție a ariilor conturilor

Anexa 9. Codul utilizat pentru urmărirea fiecărui obiect identificat

```

1  for object_id, (centroid, bbox) in object.items():
2      x, y, w, h = bbox
3      cx, cy = centroid
4      total_cars.add(object_id)
5      cv.putText(frame, f"ID {object_id}", (cx - 10, cy - 10),
        ↪ cv.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 255), 2)
6      cv.circle(frame, (cx, cy), 2, (0, 0, 255), -1)
7      cv.rectangle(frame, (x, y), (x + w, y + w), (0, 255, 0), 2)

```

Listing 9. Cum este realizată urmărirea în funcție de centroizi

Anexa 10. Bucla principală de detecție utilizând machine learning

```

1  while True:
2      ret, frame = cap.read()
3      if not ret:
4          break
5      results = model.track(frame, stream=True, persist=True)
6
7      for r in results:
8          if r.bboxes.id is None:
9              continue
10
11         boxes = r.bboxes
12         ids = boxes.id.cpu().tolist()
13         cls = boxes.cls.cpu().tolist()
14         xyxy = boxes.xyxy.cpu().numpy()
15
16
17         for box, obj_id, cls in zip(xyxy, ids, cls):
18

```



```
19         x1, y1, x2, y2 = map(int, box)
20         label = model.names[int(cls)]
21
22
23         cv.rectangle(frame, (x1, y1), (x2, y2), (255, 0, 0), 2)
24         cv.putText(frame, f"{label} ID {obj_id}", (x1, y1-10),
25                     cv.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 0), 2)
26
27         center = ((x1 + x2) // 2, (y1 + y2) // 2)
28
29         # Detection for entries in intersection
30         if obj_id not in counted_entry_ids:
31             for name, poly in ENTRY_ZONES.items():
32                 if point_intersection_with_poly(center, poly):
33                     entry_counts[name] += 1
34                     counted_entry_ids.add(obj_id)
35                     break
36         if obj_id not in counted_exit_ids:
37             for name, line in EXIT_ZONES.items():
38                 if point_cross_line(center, line):
39                     exit_counts[name] += 1
40                     counted_exit_ids.add(obj_id)
41                     break
42
43
44
45     cv.imshow("Tracked vehicle", frame)
46     out.write(frame)
47     if cv.waitKey(1) & 0xFF == ord('q'):
48         break
```

Listing 10. Bucla principală de detecție ce utilizează machine learning

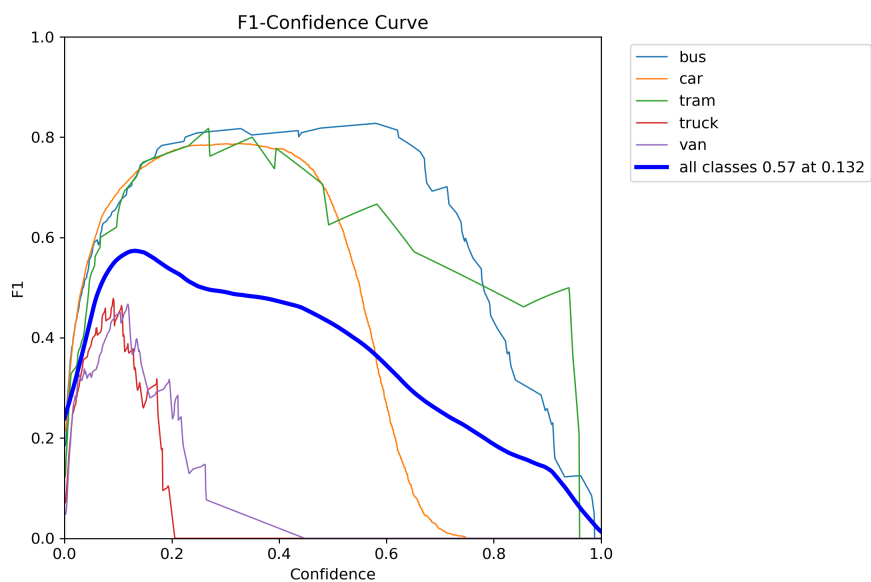
Anexa 11. Rezultatele obținute în urma antrenării

Figura A.1. Curba F1

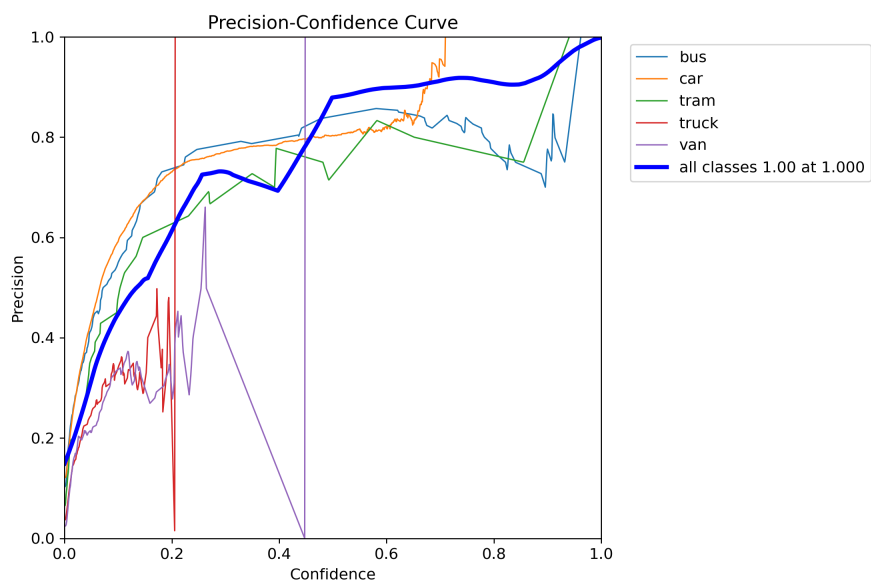


Figura A.2. Curba de precizie

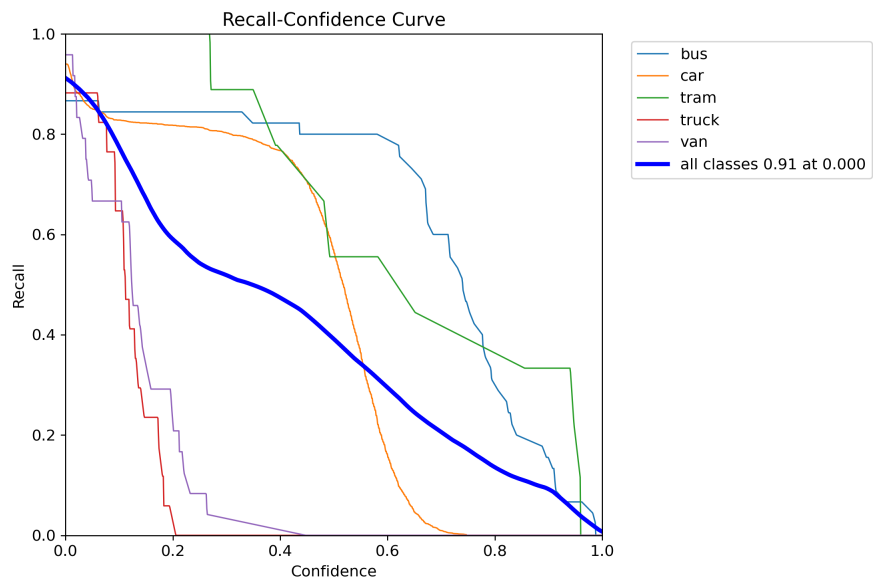


Figura A.3. Curba recall

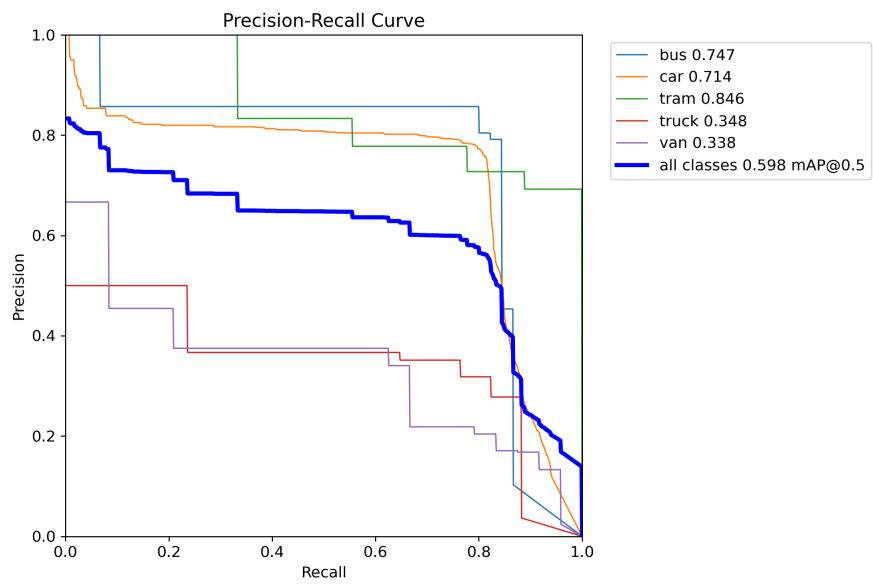


Figura A.4. Curba Precision-Recall



Figura A.5. Matricea de confuzie

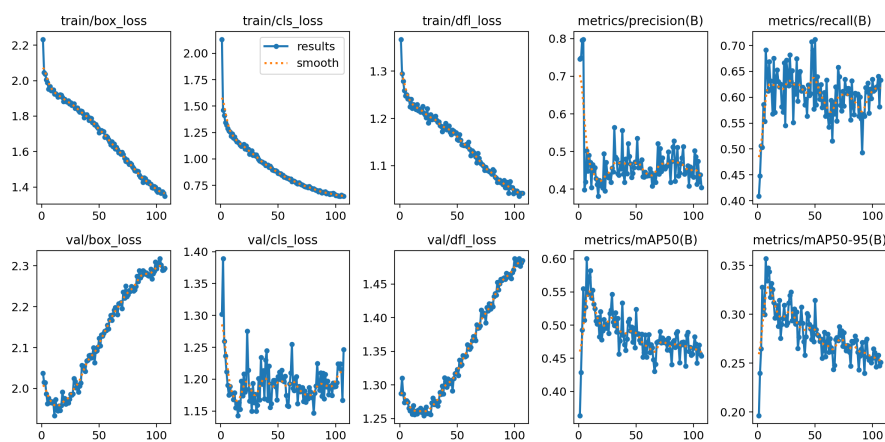


Figura A.6. Rezultatele finale

